Implement the following using FPGAs
Experiment-1: Arithmetic Logic Unit

AIM: To design Arithmetic logic Unit (ALU) using Verilog code & dump it in FPGA usingXilinx ISE Design suite and observe outputs for corresponding inputs on LEDs.

SOFTWARE USED: Xilinx ISE Design suite

PROCEDURE:

Step-1:

- Login into windows.
- Open Xilinx ISE Design suite -> new project -> project name ->

  next->Family->Artix7

  Device -> XC7A100T

  Package -> CSG324 -> next -> finish

Step-2: Verilog code

- Right click on chip -> New source -> Verilog module -> filename -> finish -> writecode
- Synthesize Tool -> Check syntax -> View RTL schematic
- Simulation -> force clock inputs -> verify output for corresponding input conditions

Step-3: FPGA dumping

- Right click on chip -> New source -> Implementation Constraints File-> file name_ucf -> next -> finish -> write ucf file code
- Implement Design -> Translate -> Map -> Place & Route -> Generate programmingfile
- Configure Target Device -> Boundary scan -> Right click ->Initialize chain -> yes ->browse bit file -> open -> click no -> ok
- Right click on chip -> program -> Verify outputs for all input conditions

VERILOG CODE:

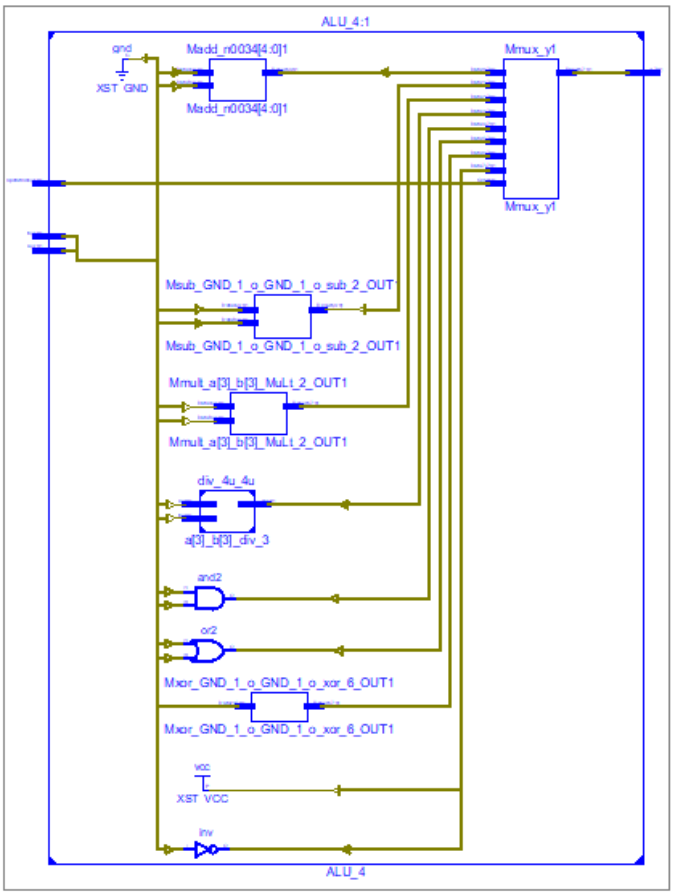
```
module ALU_4(y,a,b,operations);
 output [7:0]y;
 input  [3:0]a;
 input  [3:0]b;
 input  [2:0]operations;
 reg [7:0]y;

 always @(a,b,operations)
 begin
 case(operations)
     3'b000: y=a+b;
     3'b001: y=a-b;
     3'b010: y=a*b;
     3'b011: y=a/b;
     3'b100: y=a&b;
     3'b101: y=a|b;
     3'b110: y=a^b;
     3'b111: y=~a;
   endcase
  end
  endmodule
```

UCF FILE:

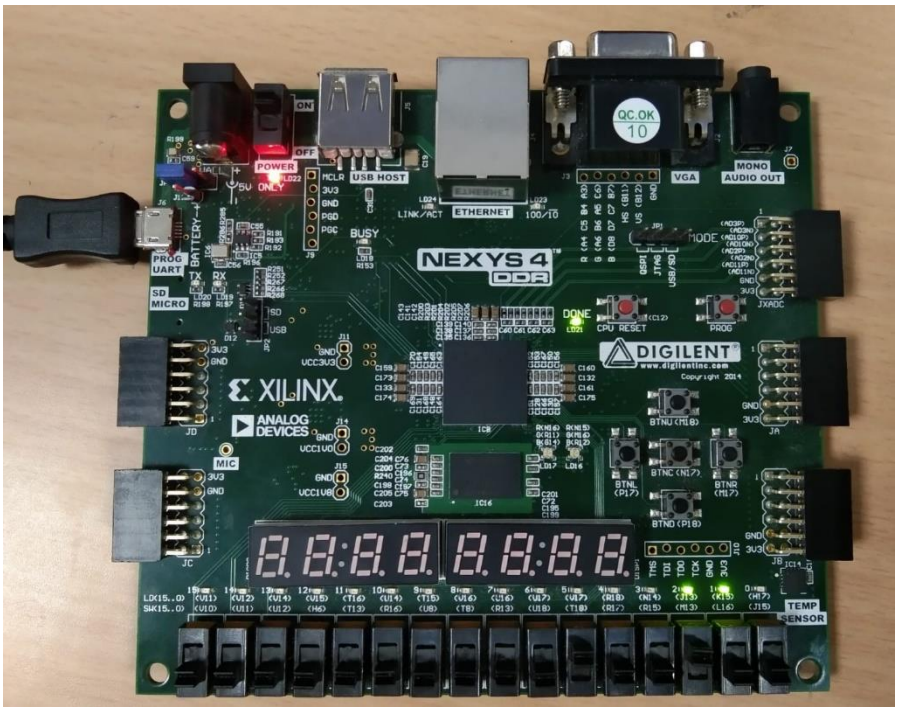
```
NET "y[0]" IOSTANDARD = LVCMOS25;
 NET "y[0]" DRIVE = 12;
 NET "y[0]" SLEW = SLOW;
 NET "y[1]" IOSTANDARD = LVCMOS25;
 NET "y[1]" DRIVE = 12;
 NET "y[1]" SLEW = SLOW;
 NET "y[2]" IOSTANDARD = LVCMOS25;
 NET "y[2]" DRIVE = 12;
 NET "y[2]" SLEW = SLOW;
 NET "y[3]" IOSTANDARD = LVCMOS25;
 NET "y[3]" DRIVE = 12;
NET "y[3]" SLEW = SLOW;
NET "y[4]" IOSTANDARD = LVCMOS25;
NET "y[4]" DRIVE = 12;
NET "y[4]" SLEW = SLOW;
NET "y[5]" IOSTANDARD = LVCMOS25;
NET "y[5]" DRIVE = 12;
NET "y[5]" SLEW = SLOW;
NET "y[6]" IOSTANDARD = LVCMOS25;
NET "y[6]" DRIVE = 12;
NET "y[6]" SLEW = SLOW;
NET "y[7]" IOSTANDARD = LVCMOS25;
NET "y[7]" DRIVE = 12;
NET "y[7]" SLEW = SLOW;
NET "a[0]" IOSTANDARD = LVCMOS25;
NET "a[1]" IOSTANDARD = LVCMOS25;
```

```
NET "a[2]" IOSTANDARD = LVCMOS25;
NET "a[3]" IOSTANDARD = LVCMOS25;
NET "b[0]" IOSTANDARD = LVCMOS25;
NET "b[1]" IOSTANDARD = LVCMOS25;
NET "b[2]" IOSTANDARD = LVCMOS25;
NET "b[3]" IOSTANDARD = LVCMOS25;
NET "operations[0]" IOSTANDARD = LVCMOS25;
NET "operations[1]" IOSTANDARD = LVCMOS25;
NET "operations[2]" IOSTANDARD = LVCMOS25;

# PlanAhead Generated physical constraints

NET "a[0]" LOC = J15;
NET "a[1]" LOC = L16;
NET "a[2]" LOC = M13;
NET "a[3]" LOC = R15;
NET "b[0]" LOC = R17;
NET "b[1]" LOC = T18;
NET "b[2]" LOC = U18;
NET "b[3]" LOC = R13;
NET "operations[0]" LOC = T8;
NET "operations[1]" LOC = U8;
NET "operations[2]" LOC = R16;
NET "y[0]" LOC = H17;
NET "y[1]" LOC = K15;
NET "y[2]" LOC = J13;
NET "y[3]" LOC = N14;
NET "y[4]" LOC = R18;
NET "y[5]" LOC = V17;
NET "y[6]" LOC = U17;
NET "y[7]" LOC = U16;
```

RTL SCHEMATIC:



OUTPUT:



RESULT: Successfully Implemented Arithmetic Logic Unit (ALU) and dumped it into FPGA

Experiment-2: Traffic Light Controller

AIM: To design Traffic Light Controller using Verilog code & dump it in FPGA using Xilinx ISE Design suite and observe outputs for corresponding inputs on LEDs.

SOFTWARE USED: Xilinx ISE Design suite

PROCEDURE:

Step-1:

- Login into windows.
- Open Xilinx ISE Design suite -> new project -> project name ->

  next->Family->Artix7

  Device -> XC7A100T

  Package -> CSG324 -> next -> finish

Step-2: Verilog code

- Right click on chip -> New source -> Verilog module -> filename -> finish -> writecode
- Synthesize Tool -> Check syntax -> View RTL schematic
- Simulation -> force clock inputs -> verify output for corresponding input conditions

Step-3: FPGA dumping

- Right click on chip -> New source -> Implementation Constraints File-> file name_ucf -> next -> finish -> write ucf file code
- Implement Design -> Translate -> Map -> Place & Route -> Generate programmingfile
- Configure Target Device -> Boundary scan -> Right click ->Initialize chain -> yes ->browse bit file -> open -> click no -> ok
- Right click on chip -> program -> Verify outputs for all input conditions

VERILOG CODE:

```
module traffic_controller_4way_design(clk,rst,North,South,East,West);
 input clk,rst;
 output reg [2:0]North,South,East,West;
 wire clk_out;
 reg [11:0] state;

 localparam s0  = 12'h001, //12'b0000_0000_0001
 s1  = 12'h002, //12'b0000_0000_0010
 s2  = 12'h004, //12'b0000_0000_0100
 s3  = 12'h008, //12'b0000_0000_1000
 s4  = 12'h010, //12'b0000_0001_0000
 s5  = 12'h020, //12'b0000_0010_0000
 s6  = 12'h040, //12'b0000_0100_0000
 s7  = 12'h080, //12'b0000_1000_0000
 s8  = 12'h100, //12'b0001_0000_0000
 s9  = 12'h200, //12'b0010_0000_0000
 s10 = 12'h400, //12'b0100_0000_0000
 s11 = 12'h800; //12'b1000_0000_0000

 reg [3:0] count;
 localparam sec15 = 4'd15,
 sec2  = 4'd2,
 sec1  = 4'd1;

clk_100Mhz_1s div1(clk,rst,clk_out);
 always @ (posedge clk_out)
 begin
 if(rst)
 begin
 state = s0;
 count = 1;
 end
 else
 begin
 case(state)
   s0 :if(count < sec15)
   begin
   state = s0;
   count = count +1;
   end
   else
   begin
   state = s1;
   count = 1;
   end
  s1 : if(count < sec2)
  begin
  state = s1;
  count = count +1;
  end
```

```
else
begin
state = s2;
count = 1;
end
s2 : if(count < sec1)
begin
state = s2;
count = count +1;
end
else
begin
state = s3;
count = 1;
end
s3 : if(count < sec15)
begin
state = s3;
count = count +1;
end
else
begin
state = s4;
count = 1;
end
s4 : if(count < sec2)
begin
state = s4;
count = count +1;
end
else
begin
state = s5;
count = 1;
end
s5 : if(count < sec1)
begin
state = s5;
count = count +1;
end
else
begin
state = s6;
count = 1;
end
s6 : if(count < sec15)
begin
state = s6;
count = count +1;
end
else
```

```
begin
state = s7;
count = 1;
end
s7 : if(count < sec2)
begin
state = s7;
count = count +1;
end
else
begin
state = s8;
count = 1;
end
s8 : if(count < sec1)
begin
state = s8;
count = count +1;
end
else
begin
state = s9
count = 1;
end
s9 : if(count < sec15)
begin
state = s9;
count = count +1;
end
else
begin
state = s10;
count = 1;
end
s10 : if(count < sec2)
begin
state = s10;
count = count +1;
end
else
begin
state = s11;
count = 1;
end
s11 : if(count < sec1)
begin
state = s11;
count = count +1;
end
else
begin
```

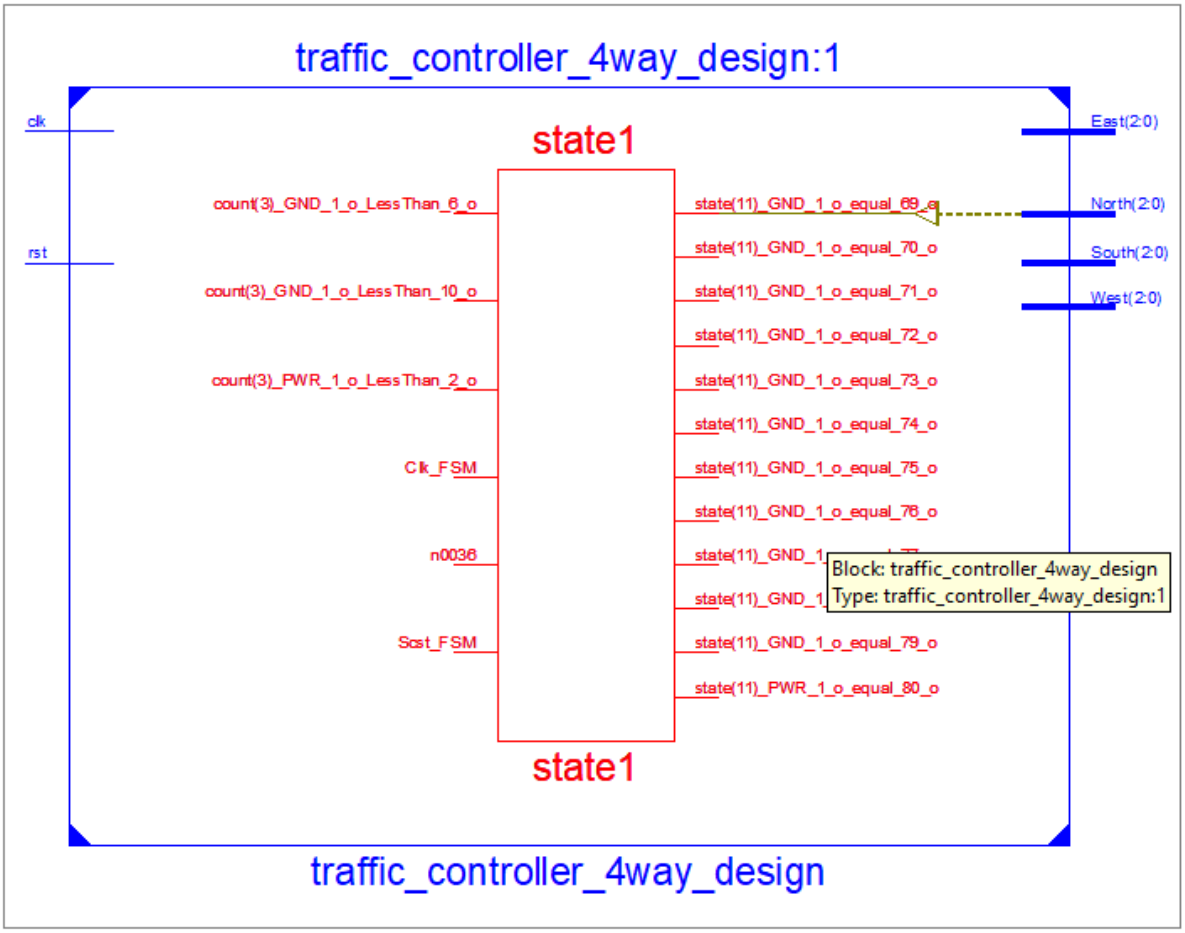
```
state = s0;
count = 1;
end
default : begin
state = s0;
count = 1;
end
endcase
end
end

always @(*)
begin //Red 001 , Yellow 010 , Green 100
 case(state)
  s0 : begin North = 3'b100; South = 3'b001 ; East = 3'b001; West = 3'b001; end
  s1 : begin North = 3'b010; South = 3'b001 ; East = 3'b001; West = 3'b001; end
  s2 : begin North = 3'b001; South = 3'b001 ; East = 3'b001; West = 3'b001; end
  s3 : begin North = 3'b001; South = 3'b100 ; East = 3'b001; West = 3'b001; end
  s4 : begin North = 3'b001; South = 3'b010 ; East = 3'b001; West = 3'b001; end
  s5 : begin North = 3'b001; South = 3'b001 ; East = 3'b001; West = 3'b001; end
  s6 : begin North = 3'b001; South = 3'b001 ; East = 3'b100; West = 3'b001; end
  s7 : begin North = 3'b001; South = 3'b001 ; East = 3'b010; West = 3'b001; end
  s8 : begin North = 3'b001; South = 3'b001 ; East = 3'b001; West = 3'b001; end
  s9 : begin North = 3'b001; South = 3'b001 ; East = 3'b001; West = 3'b100; end
  s10 : begin North = 3'b001; South = 3'b001 ; East = 3'b001; West =3'b010; end
  s11 : begin North = 3'b001; South = 3'b001 ; East = 3'b001; West =3'b001; end
 default : begin North = 3'b001; South = 3'b001 ; East = 3'b001; West = 3'b001; end
 endcase
 end
 endmodule


 module clk_100Mhz_1s(clk,rst,clk_out);
 input clk,rst;
 output clk_out;

 reg [31:0] count;
 wire [31:0] temp;

 assign temp    = (count == 32'd100000000) ? 32'd0 : count+1;
 assign clk_out = (count < 32'd50000000) ? 1'b1  : 1'b0;

 always @(posedge clk)
 begin
 if(rst)
 count =0;
 else
  count = temp;
  end
  endmodule
```

UCF FILE:

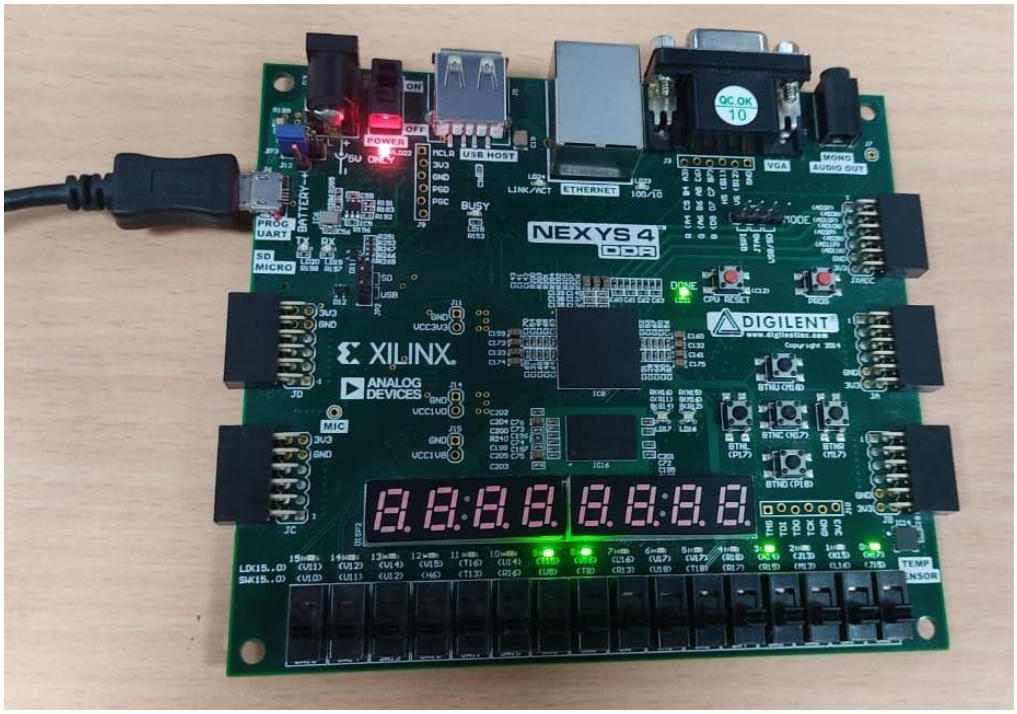
```
## Clock signal
NET "clk" LOC = E3;
NET "rst" LOC = J15;
NET "North[0]" LOC = H17;
NET "North[1]" LOC = K15;
NET "North[2]" LOC = J13;
NET "South[0]" LOC = N14;
NET "South[1]" LOC = R18;
NET "South[2]" LOC = V17;
NET "East[0]" LOC = U17;
NET "East[1]" LOC = U16;
NET "East[2]" LOC = V16;
NET "West[0]" LOC = T15;
NET "West[1]" LOC = U14;
NET "West[2]" LOC = T16;
NET "North[0]" IOSTANDARD = LVCMOS25;
NET "North[0]" DRIVE = 12;
NET "North[0]" SLEW = SLOW;
NET "North[1]" IOSTANDARD = LVCMOS25;
NET "North[1]" DRIVE = 12;
NET "North[1]" SLEW = SLOW;
NET "North[2]" IOSTANDARD = LVCMOS25;
NET "North[2]" DRIVE = 12;
NET "North[2]" SLEW = SLOW;
NET "South[0]" IOSTANDARD = LVCMOS25;
NET "South[0]" DRIVE = 12;
NET "South[0]" SLEW = SLOW;
NET "South[1]" IOSTANDARD = LVCMOS25;
NET "South[1]" DRIVE = 12;
NET "South[1]" SLEW = SLOW;
NET "South[2]" IOSTANDARD = LVCMOS25;
NET "South[2]" DRIVE = 12;
NET "South[2]" SLEW = SLOW;
NET "East[0]" IOSTANDARD = LVCMOS25;
NET "East[0]" DRIVE = 12;
NET "East[0]" SLEW = SLOW;
NET "East[1]" IOSTANDARD = LVCMOS25;
NET "East[1]" DRIVE = 12;
```

```
NET "East[1]" SLEW = SLOW;
NET "East[2]" IOSTANDARD = LVCMOS25;
NET "East[2]" DRIVE = 12;
NET "East[2]" SLEW = SLOW;
NET "West[0]" IOSTANDARD = LVCMOS25;
NET "West[0]" DRIVE = 12;
NET "West[0]" SLEW = SLOW;
NET "West[1]" IOSTANDARD = LVCMOS25;
NET "West[1]" DRIVE = 12;
NET "West[1]" SLEW = SLOW;
NET "West[2]" IOSTANDARD = LVCMOS25;
NET "West[2]" DRIVE = 12;
NET "West[2]" SLEW = SLOW;
NET "clk" IOSTANDARD = LVCMOS25;
NET "rst" IOSTANDARD = LVCMOS25;
```

RTL SCHEMATIC:



OUTPUT:



RESULT: Successfully Implemented Traffic Light Controller and dumped it into FPGA.